An-Najah National University
Faculty of Engineering and Information Technology
Department of Computer Engineering

DOS

Distributed and Operating Systems

Project Report

Academic Year	2025 – 2026
Instructor	Dr. Samer Arandi
Student 1	Yahya Jaara — 12240366
Student 2	Ahmed Saif — 12217370
Submission	https://github.com/Ahmad-S20/BazarProject.git

Lab 1 — Bazar.com: A Multi-tier Online Book Store

1. Overview

Bazar.com is a small online bookstore built using a multi-tier microservices architecture. Instead of building everything as a single program, the system is split into three independent services that communicate with each other over HTTP using REST APIs. Each service runs as its own process and handles a specific part of the system's responsibility.

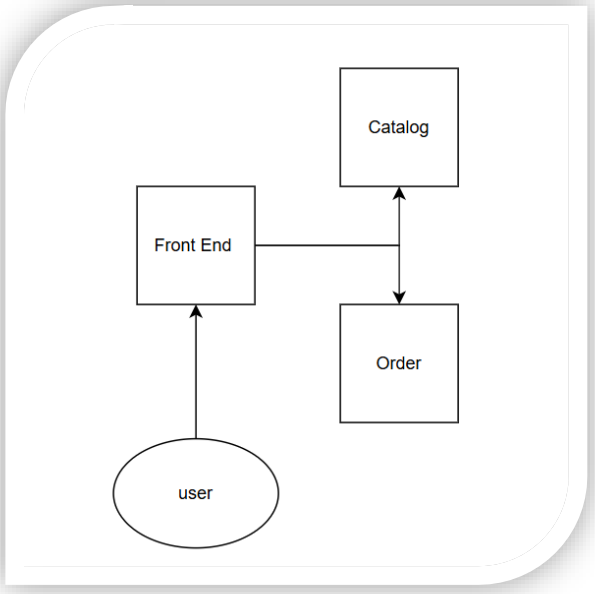
2. System Architecture

The system follows a two-tier design with a front-end tier and a backend tier. The backend is further split into two separate microservices: the catalog server and the order server.

The three services and their ports are:

- Frontend Service — port 3000: the only entry point for users. Accepts search, info, and purchase requests and forwards them to the appropriate backend service.
- Catalog Service — port 3001: manages all book data including title, topic, price, and stock. Supports search by topic, query by item, and update operations.
- Order Service — port 3002: handles purchase requests. Verifies stock availability through the catalog, decrements stock on success, and logs every order to a file.

System Architecture Diagram:



3. How It Works

3.1 Search and Info Flow

When a user sends a search or info request to the frontend, the frontend forwards it directly to the catalog service. The catalog filters through its book data and returns the matching results as a JSON response. The frontend then passes this response back to the user.

3.2 Purchase Flow

When a user sends a purchase request, the frontend forwards it to the order service. The order service then contacts the catalog to check whether the book is in stock. If it is, the order service instructs the catalog to decrease the stock by one, logs the purchase to a file called orders.txt with the book name, price, and timestamp, and returns a success message. If the book is out of stock, the purchase fails and the user gets an error message.

3.3 Data Persistence

Book data is stored in a CSV file called catalog.csv. When the catalog service starts, it reads the file into memory. Every time a stock update happens, the updated data is immediately written back to the CSV file. This means data is not lost when the server restarts.

3.4 Restocking

The catalog service includes an automatic restock timer that runs every 60 seconds and adds stock to each book. This simulates periodic arrivals of new inventory.

3.5 Concurrency

Express.js handles concurrent requests automatically using Node.js's non-blocking I/O model. No additional threading code was needed.

4. REST API Endpoints

Method	Endpoint	Description
GET	/search/:topic	Search books by topic — returns id and title of matching books
GET	/info/:id	Get full details of a book by ID (title, topic, price, stock)
POST	/purchase/:id	Purchase a book by ID — verifies stock and decrements on success
GET	/search/:topic	Catalog: filter books by topic
GET	/info/:id	Catalog: return details of a specific book
POST	/update/:id	Catalog: update stock of a book (increase or decrease)
POST	/purchase/:id	Order: full purchase flow — check, decrement, log

5. Docker

Each service is packaged in its own Docker container using a Dockerfile based on the node:18-alpine image. Docker Compose is used to start all three containers together with a single command. A key challenge was inter-container networking — inside Docker, containers cannot reach each other using localhost. This was solved by using environment variables in the JavaScript code and Docker service

names in docker-compose.yml. For example, the frontend reaches the catalog at http://catalog-service:3001 rather than http://localhost:3001.

6. How to Run

Option 1 — Without Docker

Open three terminals and run each service:

- cd catalog-service && node index.js
- cd order-service && node index.js
- cd frontend-service && node index.js

Option 2 — With Docker

- docker-compose up --build (first time or after code changes)
- docker-compose up (subsequent runs)
- docker-compose down (to stop all containers)

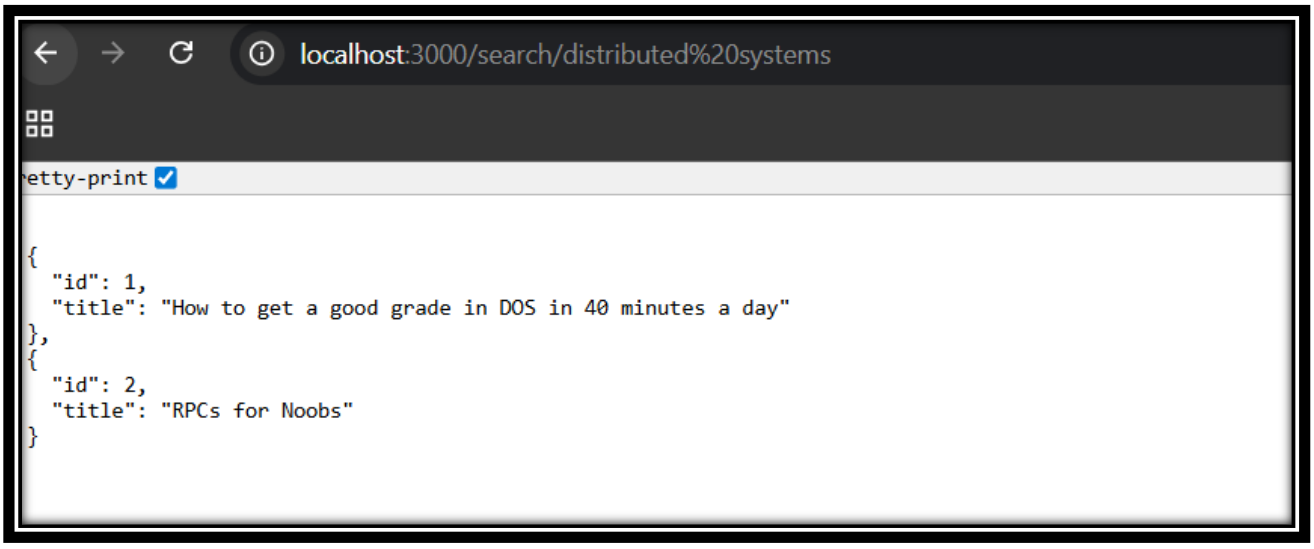
Testing

- curl http://localhost:3000/search/distributed%20systems
- curl http://localhost:3000/info/2
- curl -X POST http://localhost:3000/purchase/2

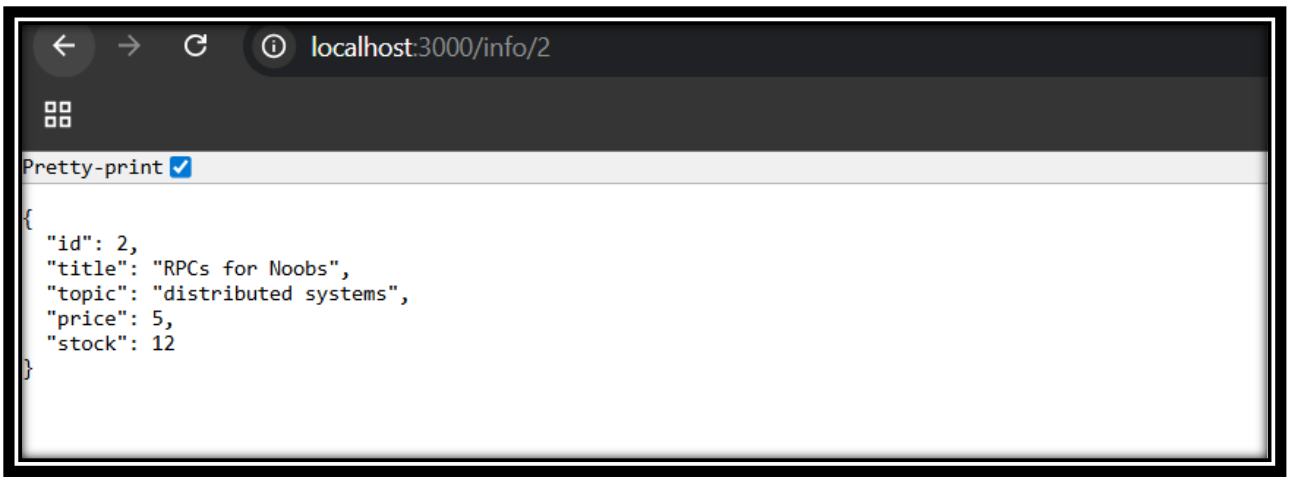
7. Program Output — Lab 1

The following section shows sample output from running the system.

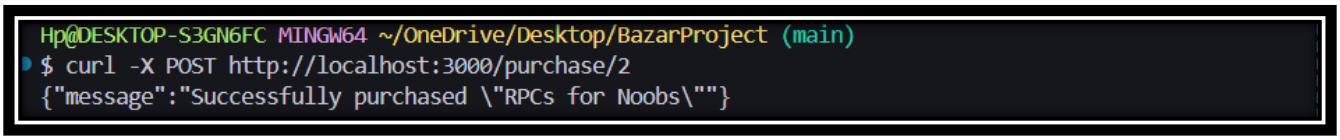
1) First let's try to search for a topic here we will search for "distributed systems"



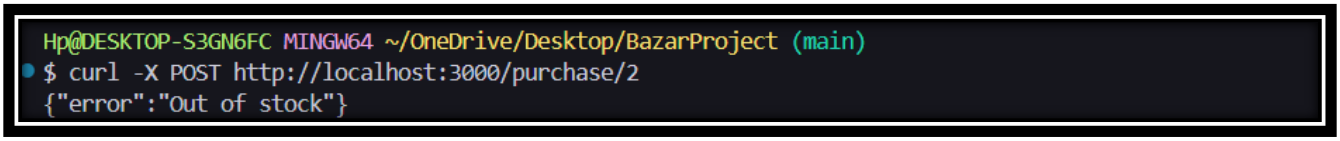
2) No search based on the ID to get the full info



3) Purchase a book with a specific ID and then search it to see if it actually worked



4) Try to purchase when the stock is zero.



As we can see we got an "Out of stock" Error.

8. Design Tradeoffs

- CSV vs Database: A CSV file was chosen over SQLite to keep things lightweight as recommended. The downside is that CSV is not ideal for concurrent writes. For this project's scale, the risk is acceptable.
- In-memory + CSV: Books are loaded into memory on startup and written back on every update. Reads are fast but a crash mid-write could cause minor data loss. A real production system would use a transactional database to prevent this.
- Fixed restock amount: The timer adds a fixed number of books every interval. A real system would have variable amounts based on supplier data.

9. Known Limitations

- If the catalog service goes down, both the order service and the frontend stop working since they both depend on it — there is no fallback.
- A race condition could occur if two users buy the last copy of a book at exactly the same time, potentially causing the stock to go negative.
- There is no user authentication — anyone can purchase any book.

10. Possible Improvements

- Replace the CSV file with SQLite to handle concurrent writes more safely.
- Add a caching layer in the frontend to reduce repeated catalog queries.
- Replicate the catalog and order services for higher availability and better load distribution.
- Add a health check endpoint to each service so failures can be detected automatically.
- Add input validation to reject invalid book IDs or malformed requests.

Lab 2 — Replication, Caching, and Consistency

1. Overview

Lab 2 builds on top of Lab 1 by adding replication, caching, and consistency mechanisms to handle a higher workload. Three new books were also added to the catalog. The motivation was that as the store grew in popularity, users started experiencing slow response times. The solution was to run multiple copies of each backend service and cache frequent read results at the frontend.

2. New Books Added

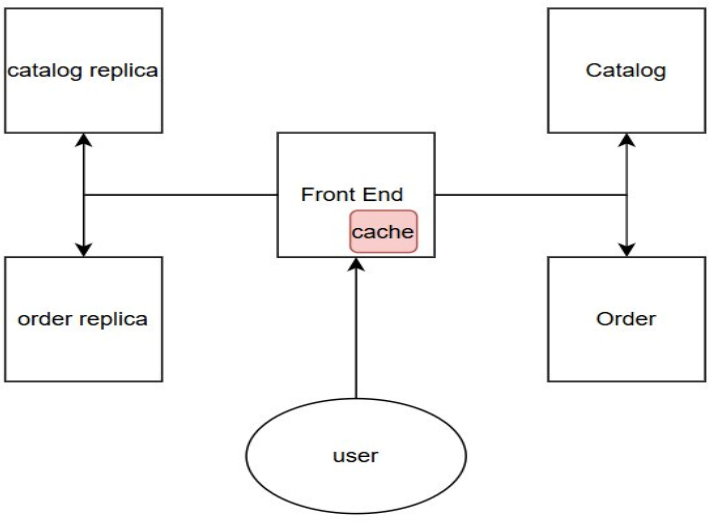
- How to finish Project 3 on time — distributed systems
- Why theory classes are so hard — undergraduate school
- Spring in the Pioneer Valley — undergraduate school

3. Updated Architecture

The system now runs five services instead of three. The catalog and order services are each replicated, giving two instances of each running simultaneously.

- Catalog Replica 1 — port 3001
- Catalog Replica 2 — port 3003
- Order Replica 1 — port 3002
- Order Replica 2 — port 3004
- Frontend Service — port 3000 (not replicated, still the single entry point)

Updated Architecture Diagram:



4. Load Balancing

The frontend uses a round-robin algorithm to distribute incoming requests between the two replicas. A simple counter alternates between replica 1 and replica 2 on each request. This spreads the workload evenly without requiring any external load balancer.

Example flow for three consecutive info requests:

- Request 1 → Catalog Replica 1 (port 3001)
- Request 2 → Catalog Replica 2 (port 3003)
- Request 3 → Catalog Replica 1 (port 3001)

5. In-Memory Cache

The frontend maintains a simple in-memory cache — a JavaScript object that stores the results of recent search and info requests. The cache works as follows:

- When a read request arrives, the frontend checks the cache first.
- If the result is found (cache hit), it is returned immediately without contacting the catalog service.
- If not found (cache miss), the request is forwarded to a catalog replica. The result is then saved in the cache for future requests.
- Write requests (purchases) always bypass the cache entirely and go directly to the order service.

6. Cache Invalidation

The cache can become stale when stock changes. Two events trigger cache invalidation:

- Purchase: when a book is bought, the order service tells the catalog to update its stock. The catalog then sends an invalidate request to the frontend, removing that book's entry from the cache. The next request for that book will hit the catalog directly and fetch the updated data.
- Restock: when the restock timer fires, the catalog adds stock to all books and sends an invalidate request to the frontend for every single book. This ensures the cache never holds outdated stock numbers after a restock.

7. Replica Synchronization

When one catalog replica updates its stock — whether from a purchase or a restock — it immediately sends a sync request to the other replica with the new stock value. This keeps both replicas in agreement at all times. Without this, the two replicas could diverge and return different stock numbers for the same book.

8. How to Run Lab 2

With Docker

- docker-compose up --build
- All five services start in the correct order automatically.

Testing Load Balancing

- curl http://localhost:3000/info/1
- curl http://localhost:3000/info/1

Watch the frontend terminal — the first request goes to replica 1 and the second to replica 2.

Testing Cache

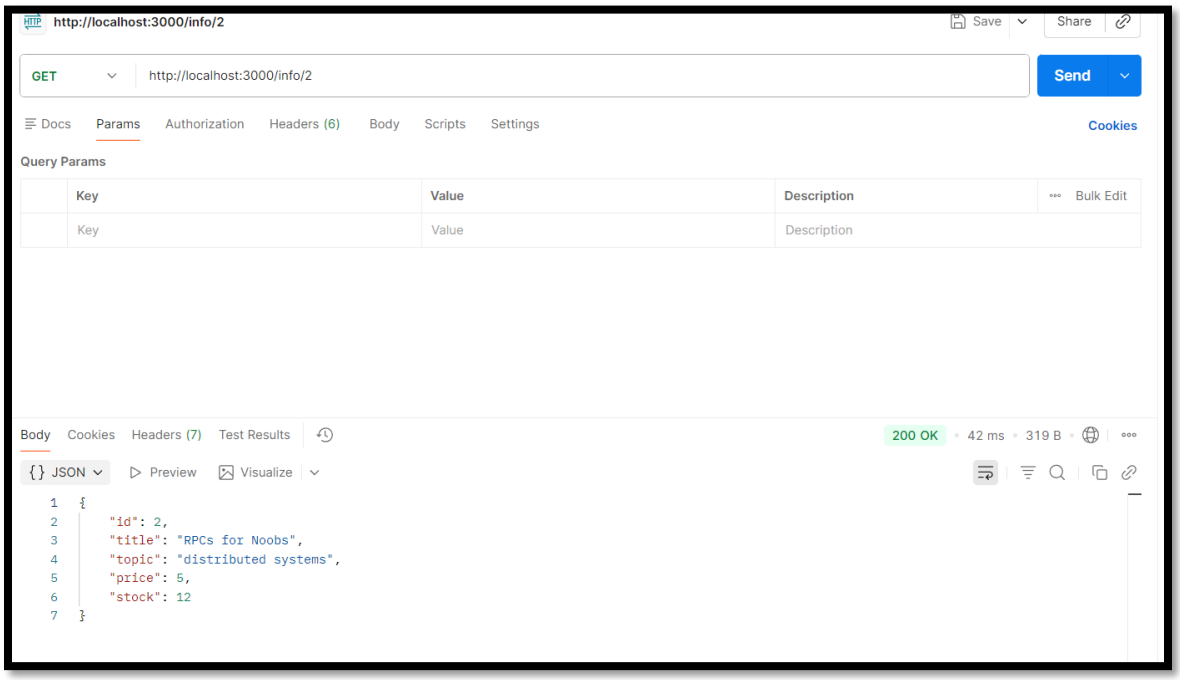
- curl http://localhost:3000/info/2 (cache MISS — fetches from catalog)
- curl http://localhost:3000/info/2 (cache HIT — returned instantly)

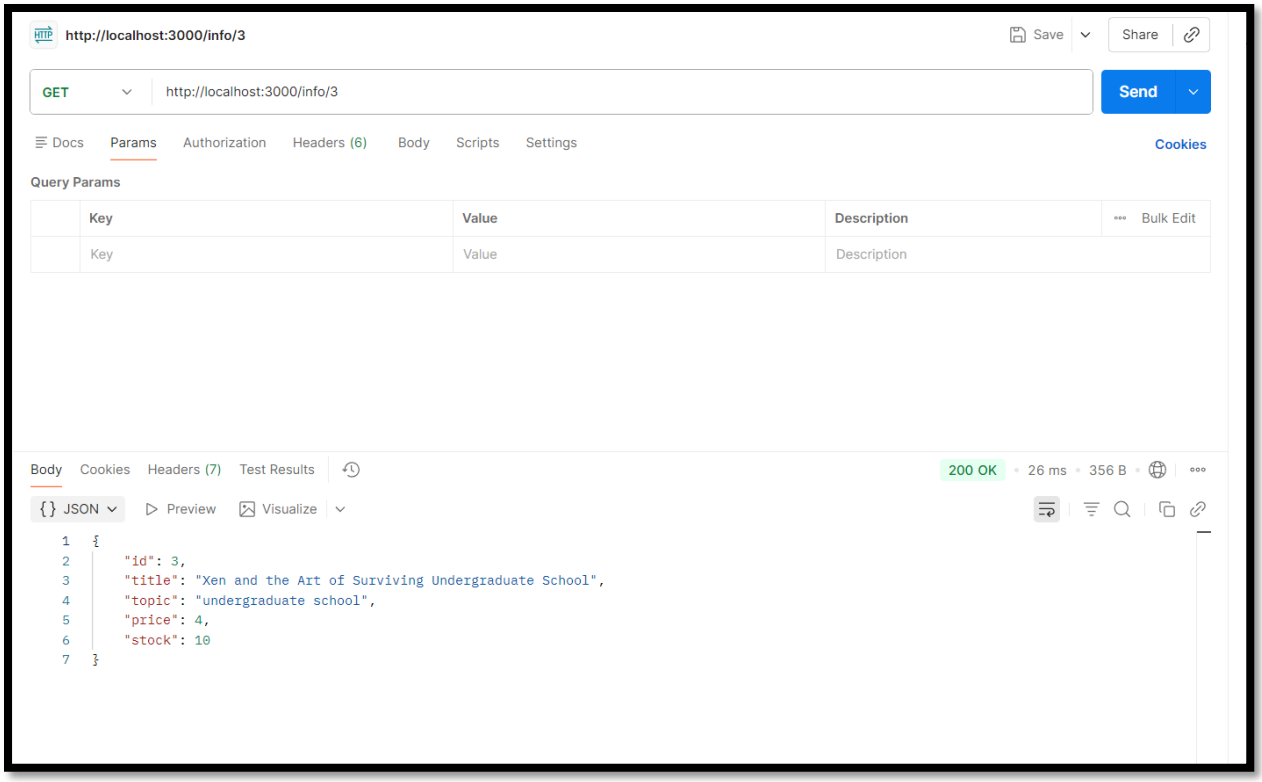
Testing Cache Invalidation

- curl http://localhost:3000/info/2
- curl -X POST http://localhost:3000/purchase/2
- curl http://localhost:3000/info/2 (cache MISS — stale entry was removed)

9. Program Output — Lab 2

1) **load balancing (requests alternating between replicas)**

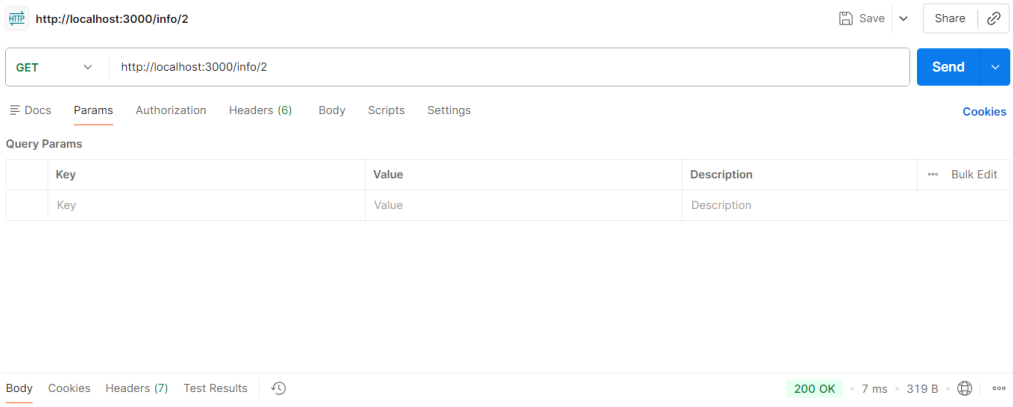


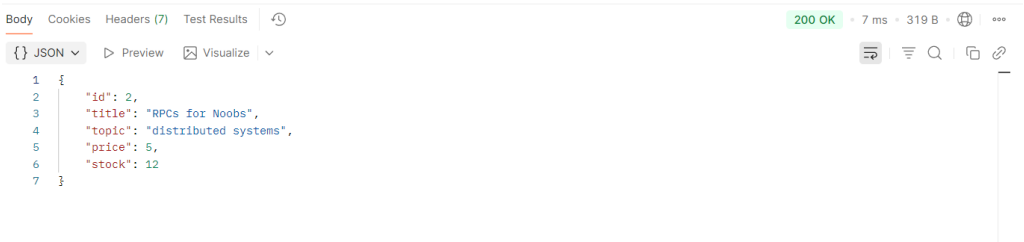



```
frontend-service-1    | Cache MISS for key: info_2
frontend-service-1    | Routing to catalog replica: http://catalog-service:3001
frontend-service-1    | Saved to cache: info_2
frontend-service-1    | Cache MISS for key: info_3
frontend-service-1    | Routing to catalog replica: http://catalog-service-2:3003
frontend-service-1    | Saved to cache: info_3
```

As we can see first, we searched for the ID = 2 and it took it from the first server (3001 port) but then when I requested ID = 3 the request went for the replica server (3003). And this is called load balancing to balance request between them.

2) cache HIT and Cache MISS

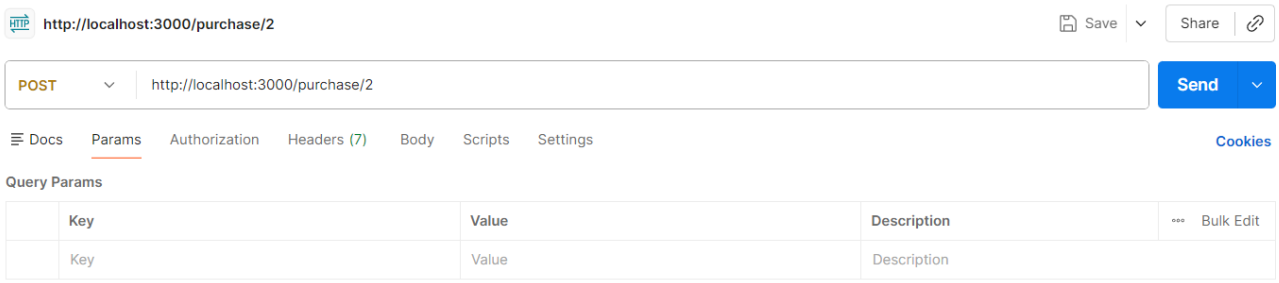


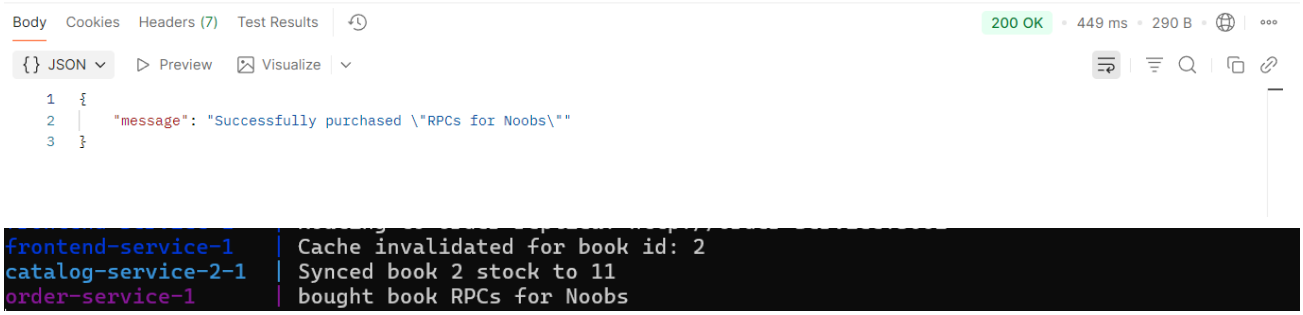


We can see the difference in the first image in the request time before it took 42ms to get info_2 but now 7ms cause it was saved to the cache and now it's a cache HIT.

Here is an example on cache miss from the previous example when we searched for info_3 it was a cache MISS and it was saved to the cache in case in the future if someone wanted to request it.

3) cache invalidation after purchase and show replica sync





Here after we purchased the book with ID 2, we had to first invalidate it from the cache (because it was saved before in it) then also sync with the other catalog server.

10. Experimental Evaluation and Measurements

10.1 Experiment 1 — Response Time with and Without Cache

We measured response times for info and search queries using Postman, which displays the exact time each request took in milliseconds. Each request type was tested multiple times and the representative values are shown below. The table compares cache miss (no cache, request goes to catalog) vs cache hit (result served from memory).

Request Type	Without Cache	With Cache	Improvement
info query	42 ms	7 ms	~83%
search query	26 ms	3 ms	~88%
purchase	449 ms	N/A	—
cache miss (subsequent)	—	~42 ms	—

Table 1: Response time comparison — cache miss vs cache hit

The results clearly show that caching reduces read request latency by 83–88%. Cache hits are served in under 10 ms because no network call is made to the catalog service. Purchase requests are not cached and take around 449 ms because they involve multiple steps: checking stock, updating the catalog, syncing replicas, and logging the order.

10.2 Experiment 2 — Cache Consistency Under Writes

To evaluate cache consistency, we ran the following sequence of operations:

- Sent GET /info/2 — cache MISS, result fetched from catalog (42 ms) and saved to cache.
- Sent GET /info/2 again — cache HIT, result served from memory (7 ms).
- Sent POST /purchase/2 — purchase triggered a stock update in the catalog, which immediately sent an invalidate request to the frontend cache.
- Sent GET /info/2 again — cache MISS, fresh data fetched from catalog (42 ms) with updated stock.

The Docker logs in Figure 14 confirm that the cache was invalidated immediately after the purchase and the replica was synced to the correct stock value. The overhead of the cache invalidation call itself was negligible — under 5 ms as observed in the logs. The subsequent request after invalidation saw a normal cache miss latency of approximately 42 ms, equivalent to an uncached request.

10.3 Performance Chart

The bar chart below summarizes the response time measurements across all request types, comparing cache miss, cache hit, and purchase latency:

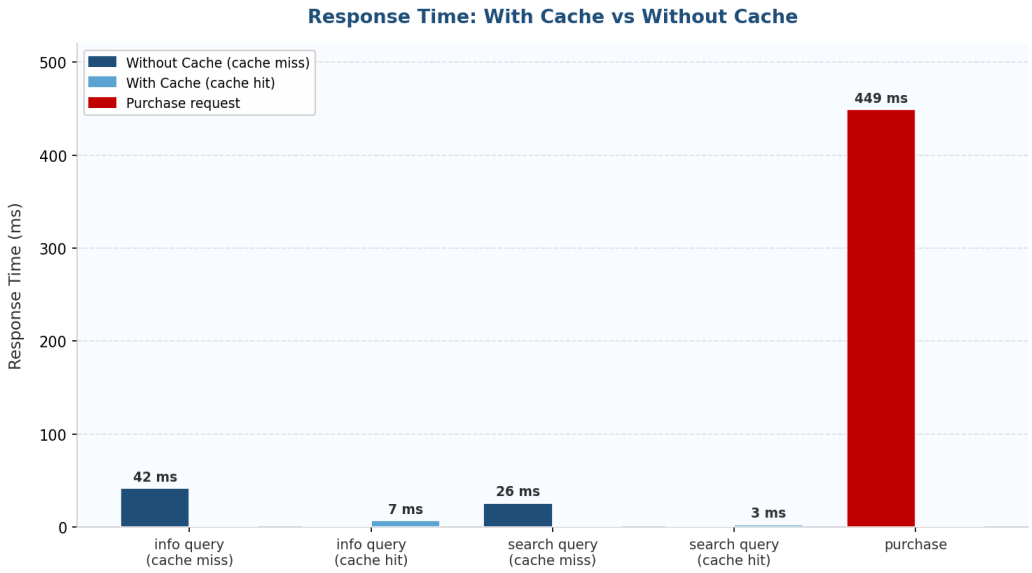


Figure 15: Response time comparison chart — cache miss vs cache hit vs purchase

As shown in the chart, cache hits are dramatically faster than cache misses for both info and search queries. The purchase bar stands out significantly because it involves a full multi-step operation across three services rather than a simple data lookup.

11. Design Tradeoffs — Lab 2

- Round-robin vs Least-loaded: Round-robin was chosen for simplicity. A least-loaded algorithm would be more efficient under uneven traffic but requires tracking request counts per replica, adding complexity.
- In-process cache vs separate cache service: The cache is embedded in the frontend rather than running as a separate microservice. This avoids extra network calls but means the cache is not shared if the frontend were ever replicated.
- Server-push invalidation vs TTL: We use server-push so the cache is invalidated the moment a write happens, giving strong consistency rather than waiting for a time-to-live expiry.
- No cache size limit: The current cache has no maximum size. In production, an LRU policy would be needed to prevent the cache from growing indefinitely.

12. Known Limitations — Lab 2

- The frontend is still a single point of failure. If it goes down, the entire system stops.
- Both catalog replicas restock independently and then sync with each other. If both restock at exactly the same time, they could send conflicting sync messages.
- The cache has no maximum size or eviction policy.
- The round-robin counter resets on restart, which could temporarily cause uneven traffic distribution.

13. Possible Improvements — Lab 2

- Replicate the frontend service and add a proper external load balancer like Nginx.
- Add a size limit to the cache with an LRU eviction policy.
- Use a distributed cache like Redis so multiple frontend instances can share the same cache.
- Add health check endpoints so Docker can detect and restart failed replicas automatically.
- Use database transactions to eliminate the race condition in concurrent stock updates.